

AdvR 01 – Names and Values

Binding Basics

Names Have Values, Not the Other Way Around

A common simplification: “the object `x` stores the value `1:3`.”

More accurate: **the name `x` is bound to the object `1:3`.**

```
x <- c(1, 2, 3)
```

Think of `<-` as an **arrow** pointing from name to value.

Multiple names can point to the same object:

```
y <- x  
obj_addr(x)
```

```
#> [1] "0x5be824d362c8"
```

```
obj_addr(y)    # same address --- no copy!
```

```
#> [1] "0x5be824d362c8"
```

Non-Syntactic Names

Syntactic names may contain letters, digits, . and _. Must start with a letter (or . not followed by a digit). Cannot be reserved words.

Non-syntactic names need backticks:

```
`_abc` <- 1  
`_abc`
```

```
#> [1] 1
```

```
`if` <- 10  
`if`
```

```
#> [1] 10
```

`make.names()` converts to syntactic names:

```
make.names(c("_abc", "if", "1st"))
```

```
#> [1] "X_abc" "if."  "X1st"
```

Exercises (Binding)

- ❶ Explain the relationship between a, b, c, and d:

```
a <- 1:10  
b <- a  
c <- b  
d <- 1:10
```

- ❷ Do these all point to the same function object? Verify with `obj_addr()`.

```
mean  
base::mean  
get("mean")  
evalq(mean)  
match.fun("mean")
```

- ❸ Why does `read.csv()` auto-converting non-syntactic names cause problems? What option suppresses it?
- ❹ Why is `.123e1` not a syntactic name?

Solutions (Binding)

- ❶ a, b, c all point to the **same object**. d points to a **different object** with the same value.

```
a <- 1:10; b <- a; c <- b; d <- 1:10  
obj_addrs(list(a, b, c, d))
```

```
#> [1] "0x5be824b2ab60" "0x5be824b2ab60" "0x5be824b2ab60"  
#> [4] "0x5be824b8df68"
```

- ❷ Yes, all five expressions return the same object:

```
obj_addrs(list(mean, base::mean, get("mean"),  
  evalq(mean), match.fun("mean"))) |> unique()
```

```
#> [1] "0x5be823580638"
```

- ❸ Column names are data; `make.names()` is non-invertible (information loss). Use `check.names = FALSE`.
❹ `.123e1` parses as the double 1.23 (dot followed by digit).

Copy-on-Modify

The Core Rule

When you modify an object with **multiple bindings**, R creates a copy:

```
x <- c(1, 2, 3)
y <- x
obj_addr(x) # both point to same object
```

```
#> [1] "0x5be824f224e8"
```

```
y[[3]] <- 4 # modifying y triggers a copy
obj_addr(y) # y now points to new object
```

```
#> [1] "0x5be824f5c968"
```

x is **unchanged** — the copy was modified, not the original.

Detecting Copies with tracemem()

`tracemem()` prints a message whenever an object is copied:

```
x <- c(1, 2, 3)
cat(tracemem(x), "\n")
#> <0x7f80c0e0ffc8>

y <- x
y[[3]] <- 4 # triggers copy
#> tracemem[0x7f80c0e0ffc8 -> 0x7f80c4427f40]

untracemem(x)
```

Turn off with `untracemem()`. Useful for finding unexpected copies in performance-sensitive code.

Copy-on-Modify in Function Calls

Function arguments create **temporary bindings** — no copy if the object is only read:

```
f <- function(a) a    # just returns a
```

```
x <- c(1, 2, 3)
```

```
z <- f(x)
```

```
obj_addr(x)
```

```
#> [1] "0x5be8251c9568"
```

```
obj_addr(z)    # same address --- no copy
```

```
#> [1] "0x5be8251c9568"
```

Inside `f()`, the parameter `a` binds to the same object as `x`. Since `f()` doesn't modify anything, no copy occurs.

Rule of thumb: Reading is free. Copying only happens on **modification**.

Shallow Copies: Lists

Lists use **shallow copies** — the list structure is copied, but elements still reference the same objects:

```
l1 <- list(1, 2, 3)
l2 <- l1
l2[[3]] <- 4
```

After modification: l2 has a new list structure, but l2[[1]] and l2[[2]] still point to the same values as l1[[1]] and l1[[2]].

Verify with `ref()`:

```
ref(l1, l2)
```

```
#> [1:0x5be82557de38] <list>
#> [2:0x5be826094eb0] <dbl>
#> [3:0x5be826094c80] <dbl>
#> [4:0x5be826094a88] <dbl>
#>
#> [5:0x5be8255e3c98] <list>
#> [2:0x5be826094eb0]
#> [3:0x5be826094c80]
#> ... [output truncated]
```

Shallow Copies: Data Frames

Data frames are lists of columns. **Column modification** is efficient:

```
d1 <- data.frame(x = c(1, 5, 6), y = c(2, 4, 3))  
d2 <- d1  
d2[, 2] <- d2[, 2] * 2  
# Only column 2 is copied; column 1 still shared
```

Row modification copies every column:

```
d3 <- d1  
d3[1, ] <- d3[1, ] * 3  
# Every column must be copied (all contain the modified row)
```

Implication: Prefer column-wise operations over row-wise for performance.

Character Vectors and the String Pool

R uses a **global string pool** — each unique string is stored once:

```
x <- c("a", "a", "abc", "d")  
ref(x, character = TRUE)
```

```
#> [1:0x5be823863b48] <chr>  
#> [2:0x5be820e76390] <string: "a">  
#> [2:0x5be820e76390]  
#> [3:0x5be8242650b0] <string: "abc">  
#> [4:0x5be82128b358] <string: "d">
```

Repeated strings point to the same location in the pool. This saves memory for character-heavy data.

Exercises (Copy-on-Modify)

- 1 Why is `tracemem(1:10)` not useful?
- 2 Why does `tracemem()` show two copies here?

```
x <- c(1L, 2L, 3L)
tracemem(x)
x[[3]] <- 4
```

- 3 Sketch the relationships:

```
a <- 1:10
b <- list(a, a)
c <- list(b, a, 1:10)
```

- 4 What happens here? Why isn't the result circular?

```
x <- list(1:10)
x[[2]] <- x
```

- 1:10 creates an object not bound to any name. It can't be modified, so tracking copies is pointless.
- Two copies** because 4 is a double but x is integer. R must:
 - Copy x (copy-on-modify, since `tracemem` holds a reference)
 - Coerce integer to double

Avoid the extra copy by assigning 4L (integer):

```
x <- c(1L, 2L, 3L)
tracemem(x)
x[[3]] <- 4L    # no type coercion needed
```

Solutions (Copy-on-Modify) — 3–4

- ③ a, b[[1]], b[[2]], and c[[2]] all reference the **same** object. c[[3]] is a **different** object with the same value. c[[1]] references b.

```
a <- 1:10; b <- list(a, a); c <- list(b, a, 1:10)
ref(c)
```

```
#> [1:0x5be824bdcaf8] <list>
#> [2:0x5be823c0eb58] <list>
#> [3:0x5be8246b77a0] <int>
#> [3:0x5be8246b77a0]
#> [3:0x5be8246b77a0]
#> [4:0x5be8246f53f8] <int>
```

- ④ Not circular because **copy-on-modify** fires. x[[2]] <- x copies the old list into a new one. The old list becomes x[[2]]; x itself is the new copy.

```
x <- list(1:10)
x[[2]] <- x
ref(x)
```

```
#> [1:0x5be824e44f48] <list>
#> [2:0x5be824ca1190] <int>
#> [3:0x5be824e6fd08] <list>
#> [2:0x5be824ca1190]
```

Object Size

Measuring with `obj_size()`

Use `lobstr::obj_size()`, not `utils::object.size()` (which ignores shared references):

```
x <- runif(1e6)
obj_size(x)
```

```
#> 8.00 MB
```

```
y <- list(x, x, x)
obj_size(y)          # only ~80 bytes more!
```

```
#> 8.00 MB
```

All three list elements reference the same vector. The list overhead is just 80 bytes (empty 3-element list).

Combined size accounts for sharing:

```
obj_size(x, y)      # same as obj_size(y)
```

```
#> 8.00 MB
```

ALTREP and String Pool

ALTREP (alternative representation) stores sequences compactly:

```
obj_size(1:3)
```

```
#> 680 B
```

```
obj_size(1:1e3)
```

```
#> 680 B
```

```
obj_size(1:1e6)
```

```
#> 680 B
```

```
obj_size(1:1e9)      # all the same!
```

```
#> 680 B
```

Only the start and end are stored — no matter how long the sequence.

String pool — each unique string stored once; repeating is cheap:

```
obj_size(rep("bananas bananas bananas", 100))
```

```
#> 928 B
```

Exercises (Object Size)

- ❶ Why are `object.size(y)` and `obj_size(y)` so different?

```
y <- rep(list(runif(1e4)), 100)
object.size(y)    # ~8 MB
obj_size(y)       # ~81 KB
```

- ❷ Why is this list's size misleading?

```
funs <- list(mean, sd, var)
obj_size(funs)
```

- ❸ Predict the output of each line (see next slide for answer).

- 1 `object.size()` doesn't account for shared references. All 100 elements point to the same vector, so true size is ~80 KB, not ~8 MB.
- 2 These functions exist in base R regardless — they're always loaded. Measuring their “size” doesn't reflect freeable memory.

③ Key predictions — initial state:

```
a <- runif(1e6)
cat("a:", obj_size(a), "\n")
```

```
#> a: 8000048
```

```
b <- list(a, a)
cat("b:", obj_size(b), "\n")      # a + 64B list
```

```
#> b: 8000112
```

```
cat("a,b:", obj_size(a, b), "\n") # same as b (shared)
```

```
#> a,b: 8000112
```

Both list elements share a → b is only 64 bytes larger than a.

Solutions (Object Size) — 3b

After modifications — sharing breaks:

```
b[[1]][[1]] <- 10 # copy-on-modify: b[[1]] diverges
cat("b:", obj_size(b), "\n")      # ~16 MB (two copies)
```

```
#> b: 16000160
```

```
cat("a,b:", obj_size(a, b), "\n") # b[[2]] still shares a
```

```
#> a,b: 16000160
```

```
b[[2]][[1]] <- 10 # b[[2]] also diverges from a
cat("b:", obj_size(b), "\n")      # still ~16 MB
```

```
#> b: 16000160
```

```
cat("a,b:", obj_size(a, b), "\n") # ~24 MB (no sharing)
```

```
#> a,b: 24000208
```

Modify-in-Place

Single Binding Optimization

If an object has only **one binding**, R modifies it in place (no copy):

```
v <- c(1, 2, 3)
# v is the only name bound to this object

v[[3]] <- 4
# Modified in place --- no copy
```

Complications:

- R tracks references as: 0, 1, or “many”
- Once “many”, it **never** goes back to 1 (even after unbinding)
- Most functions create temporary references, preventing in-place modification
- Exception: **primitive** C functions (like `[[<-]`) don't increment

Practical implication: In-place modification is hard to predict. Focus on algorithmic efficiency instead.

Environments: Always Modified in Place

Environments use **reference semantics** — never copied:

```
e1 <- rlang::env(a = 1, b = 2, c = 3)
e2 <- e1

e1$c <- 4
e2$c          # also 4! same environment
```

```
#> [1] 4
```

Environments can even contain **themselves**:

```
e <- rlang::env()
e$self <- e
ref(e)
```

```
#> [1:0x5be8259e7128] <env>
```

```
#> self = [1:0x5be8259e7128]
```

This is impossible with lists (copy-on-modify prevents circularity).

Case Study: Loop Performance

Problem: Modifying a data frame in a loop copies it every iteration:

```
x <- data.frame(matrix(runif(5 * 1e4), ncol = 5))
medians <- vapply(x, median, numeric(1))

for (i in seq_along(medians)) {
  x[[i]] <- x[[i]] - medians[[i]]
}

# Each iteration copies the ENTIRE data frame!
```

Fix: Convert to list first (uses internal C code, no extra references):

```
y <- as.list(x)
for (i in seq_along(medians)) {
  y[[i]] <- y[[i]] - medians[[i]]
}

# Much fewer copies --- list modification is efficient
x <- list2DF(y)
```

Data frame approach is $O(n^2)$; list approach is $O(n)$.

Exercises (Modify-in-Place)

- ❶ Why doesn't this create a circular list?

```
x <- list()  
x[[1]] <- x
```

- ❷ Wrap the data frame and list median-subtraction methods into functions. Use `bench::mark()` to compare. How does performance change with number of columns?
- ❸ What happens if you use `tracemem()` on an environment?

Solutions (Modify-in-Place)

- ❶ Copy-on-modify prevents it. `x[[1]] <- x` copies the old (empty) list. `x` now points to a new list containing the old one — not itself.

```
x <- list()
x[[1]] <- x
ref(x)    # x and x[[1]] have different addresses
```

```
#> [1:0x5be826eef0d0] <list>
#> [2:0x5be82619c748] <list>
```

- ❷ The data frame approach is $O(n^2)$ in columns (each iteration copies all columns). The list approach is $O(n)$. The list version wins decisively above ~300 columns.
- ❸ Error: environments are not duplicated, so tracing is not useful.

```
tracemem(new.env())
```

```
#> Error in tracemem(new.env()): 'tracemem' is not useful for promise and environment objects
```

Unbinding and the Garbage Collector

How R Frees Memory

When no name points to an object, it becomes **unreachable**:

```
x <- 1:3
x <- 2:4 # old object (1:3) is now unbound
rm(x)    # new object (2:4) is now unbound
```

The **garbage collector (GC)** periodically frees unreachable objects.

Key facts:

- GC runs **automatically** when R needs memory
- Traces from global environment + call stack to find reachable objects
- Everything else is deleted
- You rarely need to call `gc()` manually
- `gc()` is useful mainly for its memory usage report

```
gc()
```

```
#>          used (Mb) gc trigger (Mb) max used (Mb)
#> Ncells  681355 36.4    1391272 74.4    1391272 74.4
#> Vcells 5263258 40.2    10146329 77.5    6382730 48.7
```